LAPORAN PRATIKUM PEKAN 3

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR.....i

BAB I.....3

PENDAHULUAN.....3

1.1 Latar Belakang.....3

1.2 Tujuan.....3

1.3 Manfaat Praktikum.....4

BAB II.....5

PEMBAHASAN.....5

2.1 Kode Program.....**Error! Bookmark not defined.**

2.2 Langkah Kerja.....**Error! Bookmark not defined.**

2.3 Analisis Hasil.....**Error! Bookmark not defined.**

BAB III.....**Error! Bookmark not defined.**

PENUTUP.....**Error! Bookmark not defined.**

3.1 Kesimpulan.....**Error! Bookmark not defined.**

3.2 Saran.....**Error! Bookmark not defined.**

DAFTAR PUSTAKA.....**Error! Bookmark not defined.**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Progaram

a. Kode program latihanStack_2511531004

```
package pekan3_2511531004;

import java.util.Stack;

public class latihanStack_2511531004 {


    public static void main(String[] args) {

            Stack <Integer> s = new Stack <Integer> () ;

            s.push (42) ;

            s.push (-3);

            s.push (17) ;

            System.out.println("nilai stack ke " + s);

            System.out.println("nilai pop " + s.pop());

            System.out.println("nilai stack setelah pop " + s);

            System.out.println("nilai peek " + s.peek());

            System.out.println("nilai stack setelah peek " + s);

    }
```

b. Kode Program nilaiMaksimum_2511531004

```
package pekan3_2511531004;
import java.util.Stack;

public class nilaiMaksimum_2511531004 {

public static int max_2511531004 (Stack<Integer> s ) {

        Stack<Integer> backup = new Stack<Integer> ();
        int maxValue = s.pop();
        backup.push(maxValue) ;
        while( !s.isEmpty() ) {
                int next = s.pop();
                backup.push(next);
                maxValue = Math.max(maxValue, next) ;
        }
        while (!backup.isEmpty() ) {
                s.push(backup.pop())  ;

        }
        return maxValue;

}
public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer> () ;
        s.push(70);
        s.push(12);
        s.push(20);

        System.out.println("Isi stack      : " + s );
        System.out.println("Stack teratas  : " + s.peek());
        System.out.println("Nilai maksimum : " + max_2511531004(s));
```

```
    }
    }
```

c. Kode Program siswaStack_2511531004

```
package pekan3_2511531004;

import java.util.ArrayList;


class siswa_2511531004 {

    String nama;

    int nim;



    public siswa_2511531004 (String nama, int nim ) {

            this.nama = nama;

            this.nim = nim;

    }


    @Override

    public String toString() {

            return "Nim : " + nim + ", Nama : " + nama;

    }



}
```

```
public class siswaStack_2511531004 {

    private ArrayList<siswa_2511531004> stack;



    public siswaStack_2511531004 () {

            stack = new ArrayList<> ();

    }



    public void push(siswa_2511531004 mhs) {

            stack.add(mhs);

    }



    public siswa_2511531004 pop() {

            if (!isEmpty ()) {

                    return stack.remove(stack.size() - 1) ;


            }

            return null;
```

```
}

public siswa_2511531004 peek() {
        if (!isEmpty ()) {
                return stack.get(stack.size() -1 ) ;
        }

        return null;

}


public boolean isEmpty() {
        return stack.isEmpty() ;
}

public void tampilkanSiswa_2511531004 () {
        for ( int i = stack.size() - 1 ; 1>= 0; i++) {
                System.out.println(stack.get(i));
        }
}

public void main(String[] args) {
```

```
        siswaStack_2511531004 studentStack = new siswaStack_2511531004 () ;

        siswa_2511531004 mhs1 = new siswa_2511531004 ("Ali" , 1);
        siswa_2511531004 mhs2 = new siswa_2511531004 ("Boby" , 2);
        siswa_2511531004 mhs3 = new siswa_2511531004 ("Charles" , 3);


        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);


        System.out.println("siswa ada di dalam stack");
        studentStack.tampilkanSiswa_2511531004();

        System.out.println("siswa teratasa " + studentStack.peek() );
        System.out.println("mengeluarkan siswa teratas dari stackk " + studentStack.pop());
        System.out.println("daftar siswa setelah di pop ");
        studentStack.tampilkanSiswa_2511531004();
    }

}
```

d. Kode Program stackArray_2511531004

```
package pekan3_2511531004;
public class stackArray_2511531004 {
    static final int  Max = 1000;

    int top;
    int a[] =  new int[Max];

    boolean isEmpty() {

        return (top < 0 );
    }

    stackArray_2511531004()
    {
        top = -1;
    }

    boolean push_2511531004 (int x) {

        if ( top >= (Max - 1 ) ) {
            System.out.println("Stack overflow");
            return false;
```

```
        }
        else {
            a [++top] = x;
            System.out.println( x + " dimasukkan dalam stack");
            return true;
        }
    }

    int pop_2511531004 () {
        if (top < 0) {
            System.out.println("Stack underflow");
            return 0;

        }
        else {
            int x = a[top--] ;
            return x;
        }
    }

        int peek_2511531004 () {
            if (top < 0) {
                System.out.println("Stack overflow");
```

```
                return 0;
            }
            else {
                int x = a[top];
                return x;
            }
        }

        void print_2511531004 () {
            for ( int i = top; i>-1; i-- ) {
                System.out.print(" " + a[i]);
            }
        }

}
```

e. Kode program stackArrayDriver_2511531004

```
package pekan3_2511531004;

public class stackArrayDriver_2511531004 {
        public static void main(String [] args) {
            stackArray_2511531004 s = new stackArray_2511531004 ()
;
            s.push_2511531004 (10) ;
            s.push_2511531004 (20);
            s.push_2511531004 (30);
```

```
System.out.println(s.pop_2511531004() + " dikeluarkan dari stack");
System.out.println("Elemen dari atas adalah : " + s.peek_2511531004() );
System.out.print("Elemen pada stack : ");
s.print_2511531004();


    }

}
```

f. stackPostflix_2511531004

```
package pekan3_2511531004;
import java.util.Stack;
import java.util.Scanner;

public class stackPostflix_2511531004 {
    public static int postfixEvaluate_2511531004(String expression) {
        Stack<Integer> s = new Stack<Integer> ();
        Scanner input = new Scanner(expression);

        while (input.hasNext()) {
            if(input.hasNextInt()) { // an operand integer
                s.push(input.nextInt()) ;

            } else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
```

```
                    s.push(operand1 + operand2) ;

                } else if ( operator.equals("-")) {
                    s.push(operand1 - operand2);

                } else if ( operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2) ;
                }


            }



        }
        input.close();
        return s.pop () ;

    }


    public static void main (String[] args ) {


        System.out.println("Hasil dari post fix = " + postfixEvaluate_2511531004 (" 5 2 4 * + 7 -") );
    }


}
```

2.2 Langkah Kerja

a) latihanStack_2511531004

Deklarasi Stack

- Program membuat sebuah objek Stack<Integer> bernama s.
- Stack adalah struktur data dengan prinsip **LIFO (Last In, First Out)**, artinya elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Operasi Push (Menambahkan Elemen)

- s.push(42); → Menambahkan angka 42 ke dalam stack.
- s.push(-3); → Menambahkan angka -3 ke dalam stack.
- s.push(17); → Menambahkan angka 17 ke dalam stack.
- Setelah tiga kali push, isi stack adalah [42, -3, 17].

Menampilkan Isi Stack

- System.out.println("nilai stack ke " + s);
- Menampilkan isi stack saat ini: [42, -3, 17].

Operasi Pop (Menghapus Elemen Teratas)

- s.pop(); → Menghapus dan mengembalikan elemen teratas (17).
- Setelah pop, isi stack menjadi [42, -3].

Operasi Peek (Melihat Elemen Teratas)

- s.peek(); → Mengembalikan elemen teratas tanpa menghapusnya (yaitu -3).
- Isi stack tetap [42, -3].

b) nilaiMaksimum_2511531004

Deklarasi Kelas dan Metode

- Program mendefinisikan kelas nilaiMaksimum_2511531004.
- Di dalamnya terdapat metode statis max_2511531004(Stack<Integer> s) yang berfungsi untuk mencari nilai maksimum dari elemen-elemen dalam stack.

Mekanisme Pencarian Nilai Maksimum

- **Membuat Stack Backup** Sebuah stack baru bernama backup dibuat untuk menyimpan sementara elemen-elemen yang diambil dari stack utama.
- **Inisialisasi Nilai Maksimum** Elemen pertama diambil dari stack s menggunakan pop(), lalu disimpan ke maxValue. Elemen ini juga dimasukkan ke dalam backup.
- **Iterasi Elemen Stack** Selama stack s tidak kosong:
    - Ambil elemen berikutnya dengan pop().
    - Simpan elemen tersebut ke dalam backup.
    - Bandingkan dengan maxValue menggunakan Math.max(). Jika lebih besar, perbarui nilai maksimum.

Mengembalikan Elemen ke Stack Asal

- Setelah semua elemen dipindahkan ke `backup`, dilakukan proses pengembalian ke stack `s` dengan `push()` agar isi stack tetap sama seperti semula.
- **Mengembalikan Nilai Maksimum** Metode mengembalikan nilai maksimum yang ditemukan.

Metode `main`

- Membuat stack `s` bertipe `Integer`.
- Menambahkan tiga elemen: `70`, `12`, dan `20` menggunakan `push()`.
- Menampilkan isi stack: `[70, 12, 20]`.

- Menampilkan elemen teratas dengan `peek()`: `20`.
- Memanggil metode `max_2511531004(s)` untuk mencari nilai maksimum, hasilnya `70`.

c) siswaStack_2511531004

Deklarasi Kelas siswa_2511531004

- Kelas ini merepresentasikan data seorang siswa dengan atribut:
  - nama (String),
  - nim (int).
- Konstruktor digunakan untuk menginisialisasi nilai atribut saat objek dibuat.

Deklarasi Kelas siswaStack_2511531004

- Kelas ini berfungsi sebagai implementasi **stack** berbasis ArrayList<siswa_2511531004>.
- Atribut:
  - stack → wadah untuk menyimpan objek siswa.
- Metode yang tersedia:
  - push(siswa_2511531004 mhs) → menambahkan siswa ke dalam stack.
  - pop() → menghapus dan mengembalikan siswa teratas dari stack.
  - peek() → melihat siswa teratas tanpa menghapusnya.
  - isEmpty() → memeriksa apakah stack kosong.
  - tampilkanSiswa_2511531004() → mencetak isi stack dari atas ke bawah. ⚠ *Catatan*: pada kode terdapat kesalahan di kondisi loop (for (int i = stack.size()-1; 1>=0; i++)), seharusnya i >= 0.

Metode main

- Membuat objek studentStack dari kelas siswaStack_2511531004.
- Membuat tiga objek siswa:
  - mhs1 = ("Ali", 1)
  - mhs2 = ("Boby", 2)
  - mhs3 = ("Charles", 3)
- Menambahkan ketiga siswa ke dalam stack dengan push().
- Menampilkan isi stack dengan tampilkanSiswa_2511531004().
- Menampilkan siswa teratas dengan peek().
- Menghapus siswa teratas dengan pop().
- Menampilkan isi stack setelah operasi pop().

d) stackArray_2511531004

Deklarasi Kelas dan Atribut

- Kelas stackArray_2511531004 merepresentasikan struktur data **stack** menggunakan **array**.
- Atribut:
  - Max = 1000 → kapasitas maksimum stack.
  - top → penunjuk indeks elemen teratas dalam stack.
  - a[] → array bertipe int sebagai wadah elemen stack.

Konstruktor

- stackArray_2511531004() → menginisialisasi top = -1, artinya stack kosong.

Metode isEmpty()

- Mengecek apakah stack kosong dengan kondisi top < 0.
- Mengembalikan true jika kosong, false jika ada elemen.

Metode push_2511531004(int x)

- Menambahkan elemen ke dalam stack.

- Jika top >= Max - 1, maka stack penuh (**overflow**) → elemen tidak bisa ditambahkan.
- Jika masih ada ruang:
    - top dinaikkan (++top).
    - Elemen x dimasukkan ke posisi a[top].
    - Menampilkan pesan bahwa elemen berhasil dimasukkan.

Metode pop_2511531004()

- Menghapus elemen teratas dari stack.
- Jika top < 0, maka stack kosong (**underflow**) → tidak ada elemen yang bisa dihapus.
- Jika ada elemen:
    - Ambil elemen a[top].
    - Turunkan top (top--).
    - Kembalikan nilai elemen yang dihapus.

Metode peek_2511531004()

- Melihat elemen teratas tanpa menghapusnya.
- Jika stack kosong (top < 0), tampilkan pesan error.
- Jika ada elemen, kembalikan nilai a[top].

Metode print_2511531004()

- Mencetak seluruh isi stack dari atas ke bawah.
- Menggunakan perulangan for (int i = top; i > -1; i--) untuk menampilkan elemen.

e) stackArrayDriver_2511531004

Inisialisasi Stack

- Program membuat objek s dari kelas stackArray_2511531004.
- Pada saat inisialisasi, top = -1, artinya stack masih kosong.

Operasi Push

- s.push_2511531004(10); → Menambahkan angka 10 ke dalam stack.
- s.push_2511531004(20); → Menambahkan angka 20 ke dalam stack.
- s.push_2511531004(30); → Menambahkan angka 30 ke dalam stack.

- Setelah tiga kali push, isi stack adalah [10, 20, 30] (30 berada di posisi teratas).

Operasi Pop

- s.pop_2511531004(); → Menghapus dan mengembalikan elemen teratas (30).
- Output: 30 dikeluarkan dari stack.
- Isi stack sekarang [10, 20].

Operasi Peek

- s.peek_2511531004(); → Melihat elemen teratas tanpa menghapusnya.
- Hasil: 20.
- Output: Elemen dari atas adalah : 20.

Menampilkan Isi Stack

- s.print_2511531004(); → Mencetak isi stack dari atas ke bawah.
- Output: 20 10.

g. stackPostflix_2511531004

Deklarasi Kelas dan Metode

- Program mendefinisikan kelas stackPostflix_2511531004.
- Di dalamnya terdapat metode statis postfixEvaluate_2511531004(String expression) yang berfungsi untuk menghitung nilai ekspresi aritmatika dalam bentuk **postfix (Reverse Polish Notation)**.

Inisialisasi Stack dan Scanner

- Membuat objek Stack<Integer> s untuk menyimpan operand sementara.
- Membuat objek Scanner input untuk membaca token (angka atau operator) dari string ekspresi yang diberikan.

Proses Evaluasi Ekspresi

- Selama masih ada token dalam ekspresi:
    - Jika token berupa **integer (operand)** → masukkan ke dalam stack dengan push().
    - Jika token berupa **operator (+, -, , /)*:
        - Ambil dua operand teratas dari stack dengan pop().
        - Lakukan operasi sesuai operator:

        - + → penjumlahan,
        - - → pengurangan,
        - * → perkalian,
        - / → pembagian.
    - Hasil operasi dimasukkan kembali ke dalam stack dengan push().

Metode main

- Memanggil postfixEvaluate_2511531004("5 2 4 * + 7 -").
- Ekspresi tersebut berarti:

  Code

  5 + (2 * 4) - 7
- Langkah perhitungan:
    - 2 * 4 = 8
    - 5 + 8 = 13
    - 13 - 7 = 6
- Output: Hasil dari post fix = 6.

BAB III

PENUTUP

3.1 Kesimpulan

Keseluruhan program memperlihatkan pemahaman bertahap mengenai **struktr data stack** dalam bahasa Java. Dimulai dari implementasi dasar menggunakan library bawaan (latihanStack) untuk operasi push, pop, dan peek, kemudian berkembang ke algoritma pencarian nilai maksimum (nilaiMaksimum) yang memanfaatkan stack tambahan agar data tetap utuh. Selanjutnya, konsep stack diperluas untuk menyimpan objek siswa (siswaStack), sehingga tidak hanya terbatas pada tipe data primitif. Program stackArray dan stackArrayDriver menunjukkan bagaimana stack dapat diimplementasikan secara manual menggunakan array, memberikan gambaran internal cara kerja stack tanpa bantuan library. Puncaknya, stackPostflix mengaplikasikan stack dalam evaluasi ekspresi postfix, yang merupakan contoh nyata penggunaan stack dalam algoritma pemrosesan ekspresi matematika. Secara keseluruhan, keenam program ini menegaskan prinsip **LIFO (Last In, First Out)** dan menunjukkan progres belajar dari konsep dasar hingga penerapan algoritmik yang lebih kompleks.

3.2 Saran

Perbaikan Teknis

- Perbaiki loop pada siswaStack agar dapat mencetak isi stack dengan benar.
- Tambahkan validasi agar tidak terjadi operasi pop() pada stack kosong.
- Tambahkan penanganan kesalahan seperti pembagian dengan nol pada program postfix.

Pengembangan Fitur

- Tambahkan operator tambahan pada evaluasi postfix (%, ^).
- Tambahkan fitur update data siswa pada siswaStack.

- Buat penyimpanan data ke file (.txt atau .csv) agar data tidak hilang setelah program ditutup.

Pemformatan Output

- Gunakan format tampilan yang lebih rapi, misalnya tabel atau garis pemisah.
- Konsistensi pesan output agar lebih mudah dipahami.

Pengembangan Lebih Lanjut

- Kembangkan aplikasi stack ke antarmuka grafis (GUI) menggunakan Java Swing atau JavaFX agar lebih interaktif.
- Pisahkan logika ke dalam kelas layanan (misalnya StackService) agar kode lebih modular dan mudah dipelihara.

DAFTAR PUSTAKA

Oracle. (2024). *Java™ Platform, Standard Edition Documentation*. Oracle. https://docs.oracle.com/javase/8/docs/api/ (docs.oracle.com in Bing)

Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program (Early Objects)* (11th ed.). Pearson Education.

Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures* (11th ed.). Pearson.

Wirth, N. (2006). *Algorithms and Data Structures*. Springer. (Referensi umum untuk konsep struktur data seperti stack dan array).

GeeksforGeeks. (2025). *Java Stack Class – Methods and Examples*. https://www.geeksforgeeks.org/stack-class-in-java/ (geeksforgeeks.org in Bing)

TutorialsPoint. (2025). *Data Structures – Stack*. https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm (tutorialspoint.com in Bing)

Baeldung. (2025). *Guide to the Java Stack*. https://www.baeldung.com/java-stack

Tugas Struktur Data Pekan 3

a. Kode Program

Website_2511531004

```
package Struktur_data;

public class Website_2511531004 {
    // Atribut dengan akhiran 4 digit NIM
    private String judul_1004;
    private String url_1004;

    // Constructor
    public Website_2511531004(String judul_1004, String url_1004) {
        this.judul_1004 = judul_1004;
        this.url_1004 = url_1004;
    }

    // Getter
    public String getJudul_1004() {
        return judul_1004;
    }

    public String getUrl_1004() {
        return url_1004;
    }

    // Setter
    public void setJudul_1004(String judul_1004) {
        this.judul_1004 = judul_1004;
    }

    public void setUrl_1004(String url_1004) {
```

```
        this.url_1004 = url_1004;
    }

    // Representasi String
    @Override
    public String toString() {
        return "Judul: " + judul_1004 + ", URL: " + url_1004;
    }
}
```

2. Browser_2511531004

```
package Struktur_data;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511531004 {
    public static void main(String[] args) {
        Stack<Website_2511531004> history = new Stack<>();
        Scanner sc = new Scanner(System.in);
        int pilihan;

        do {
            System.out.println("=== Browser History NIM: 2511531004 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan = sc.nextInt();
```

```
sc.nextLine(); // membersihkan buffer

switch (pilihan) {
    case 1:
        System.out.print("Masukkan Judul: ");
        String judul = sc.nextLine();
        System.out.print("Masukkan URL: ");
        String url = sc.nextLine();

        history.push(new Website_2511531004(judul, url));
        System.out.println("Berhasil mengunjungi halaman!");
        break;

    case 2:
        if (!history.isEmpty()) {
            Website_2511531004 removed = history.pop();
            System.out.println("Kembali dari halaman: " +
removed.getJudul_1004());
        } else {
            System.out.println("History kosong, tidak ada halaman untuk
kembali.");
        }
        break;

    case 3:
        if (!history.isEmpty()) {
            Website_2511531004 current = history.peek();
            System.out.println("Halaman aktif: " + current);
        } else {
            System.out.println("Tidak ada halaman aktif.");
        }
        break;
```

```
            case 4:
                if (history.isEmpty()) {
                    System.out.println("History kosong.");
                } else {
                    System.out.println("Jumlah riwayat tersimpan: " +
history.size());
                }
                break;

            case 5:
                System.out.println("Keluar dari program...");
                break;

            default:
                System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan != 5);

        sc.close();
    }
}
```